

Go Back

Hardware

Portenta Vision Shield

Tutorials

Blob Detection with Portenta and OpenMV

Saving Bitmap Camera Images to the SD Card

Connect Portenta Vision Shield to TTN with LoRa® Technology

Creating a Basic Face Filter With OpenMV

Training a Custom Machine Learning Model for Portenta H7

Using the Portenta Vision Shield - Ethernet with Arduino IDE

Using the Portenta Vision Shield - Ethernet with OpenMV

Getting Started With the Portenta Vision Shield Camera

Voice Commands With The Arduino Speech Recognition Engine

Connecting to The Things Network Using OpenMV

Portenta Vision Shield User Manual

Home / Hardware / Portenta Vision Shield / Getting Started With the Portenta Vision Shield Camera

Getting Started With the Portenta Vision Shield Camera

This tutorial shows you how to capture frames from the Portenta Vision Shield Camera module and visualize the video output through a Processing sketch.

Author: Lenard George, Sebastian Romero · Last revision: 09/25/2024

ON THIS PAGE

Overview

- Goals +
- Instructions +
- Conclusion +

Overview

This tutorial shows you how to capture frames from the Arduino Portenta Vision Shield Camera module and visualize the video output through a Processing sketch.

Goals

- Capturing the frames from the camera
- Sending the frames as a byte stream through a Serial connection
- Visualising the frames in Processing

Required Hardware and Software

Help

Portenta Vision Shield ([LoRa](#)
or [Ethernet](#))

[USB-C® cable](#) (X1)

[Arduino IDE 1.8.10+](#) or

[Arduino IDE 2.0+](#)

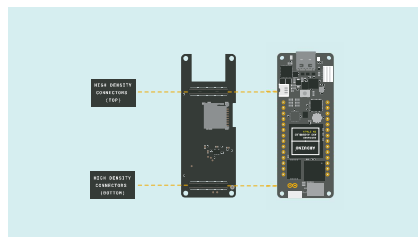
Processing 3.5.4+

Instructions

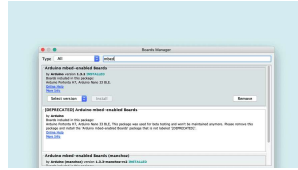
Accessing the Portenta Vision Shield's camera data is done with the help of both Arduino and the Processing IDE. The Arduino sketch handles the capture of image data by the on-board camera, while the java applet created with Processing helps to visualize this data with the help of a serial connection. The following steps will run you through how to capture, package the data through the serial port and visualize the output in Processing.

1. The Basic Setup

Connect the Portenta Vision Shield to your Portenta H7 as shown in the figure. The top and bottom high density connectors are connected to the corresponding ones on the underside of the H7 board. Plug in the H7 to your computer using the USB-C® cable.



Open the board manager in the Arduino IDE and install the latest



Download the mbed core

2. Capturing the Frames

Create a new Arduino sketch called

`CameraCaptureRawBytes.ino`.

To capture the frames you will need to use the functions contained in `camera.h` which comes with the Portenta core. This library contains all APIs related to frame capturing, motion detection and pattern recognition. Include the header file in your sketch.

```
1 #include "camera.h"
2 //For the Vision Shield
3 //#include "himax.h" //
4
5 //For the Vision Shield
6 #include "hm0360.h" // A
```



Left uncommented the library of your Vision Shield version.

Next, let's initialize a camera object and a frame buffer of the size 320*240 (76'800 bytes).

```
1 //HM01B0 himax; // for V
2
3 HM0360 himax; // for Vis
4
5 Camera cam(himax);
6 #define IMAGE_MODE CAMERA_QVGA
7 FrameBuffer fb(320,240,2);
8
9 unsigned long lastUpdate;
```

In the `setup()` function, let's start the Serial communication at `921600` baud rate and initialize the camera using `cam.begin()`.

```
1 void setup() {
2   Serial.begin(921600);
3   //Init the cam QVGA, 320x240
4   cam.begin(CAMERA_R320X240);
5 }
```

In the loop you need to capture each Frame and send it over a serial connection to the Processing sketch that will display the frames. You will use the

```
grab(uint8_t *buffer, uint32_t
timeout=5000);
```

function to fetch the frame from the frame buffer and save it into your custom data buffer.

```
1 void loop() {
2   // put your main code
3   if(!Serial) {
4     Serial.begin(921600);
5     while(!Serial);
6   }
7
8   // Time out after 2 s
9   bool timeoutDetected = false;
10
11   // Wait until the rec
12   // that they are read
13   if(!timeoutDetected &
14
15   lastUpdate = millis()
16
17   // Grab frame and writ
18   if (cam.grabFrame(fb,
19     Serial.write(fb.get
20   }
21 }
```

3. Create the Processing Sketch

Open a new processing sketch file and name it `CameraCapture.pde`.



Create a processing sketch

Let's start by importing the libraries and initializing the variables you will need to process. To process the data sent by the Portenta Vision Shield, you will need to import the following libraries:

`processing.serial.*`: a [Serial Library](#) that is used to read and write data to

`java.nio.ByteBuffer`: a
java class that provides
access to operations on byte
buffers

```
1 import processing.serial
2 import java.nio.ByteBuff
```

Next, you can initialize the
following variables to process the
received pixels from the serial port.
You can set the dimensions, pixel
count and bytes required per
frame.

```
1 // must match resolution
2 final int cameraWidth =
3 final int cameraHeight =
4 final int cameraBytesPer
5 final int cameraPixelCou
6 final int bytesPerFrame
```

To receive the frames, you will need
a Serial port, a PImage object and
an array to store the pixel values of
the frame. Add the following
variables to the code.

```
1 Serial myPort;
2 PImage myImage;
3 byte[] frameBuffer = new
4 int pixelPosition = 0;
5 int lastUpdate = 0;
6 boolean shouldRedraw = f
```

Here, you will establish a
connection to the serial port and
prepare the buffer to store the

from Processing to let it know that it is ready to receive data.

```
1 void setup() {  
2   size(640, 480);  
3  
4   // if you know the se  
5   //myPort = new Serial  
6   //myPort = new Serial  
7   myPort = new Serial(t  
8  
9   // Set the number of l  
10  myPort.buffer(bytesPe  
11  
12  // Create an image ba  
13  myImage = createImage  
14  
15  // Let the Arduino sk  
16  myPort.write(1);  
17 }
```

The draw function checks if the connection is still alive and if there is any new data that can be drawn as an image. In that case, the original image gets copied into a new image object so that it can be scaled up.

```
1 void draw() {  
2   // Time out after 1.5  
3   if(millis() - lastUpd  
4     println("Connection  
5     myPort.clear();  
6     myPort.write(1);  
7   }  
8  
9   if(shouldRedraw){  
10    PImage img = myImag  
11    img.resize(640, 480  
12    image(img, 0, 0);  
13    shouldRedraw = fals  
14  }  
15 }
```

4. Visualizing the Frames

For this step, you will use the `serialEvent()` callback function to update the `myImage` when a new data is received on the serial port.

```
1 void serialEvent(Serial
2   lastUpdate = millis(
3
4   // read the received
5   myPort.readBytes(fra
6
7   // Access raw bytes
8   ByteBuffer bb = Byte
9
10  int i = 0;
11
12  while (bb.hasRemaini
13    // read 8-bit pixe
14    byte pixelValue =
15
16    // set pixel color
17    myImage.pixels[i++
18  }
19
20  myImage.updatePixels
21
22  // Ensures that the
23  shouldRedraw = true;
24
25  // Let the Arduino s
26  // and are ready for
27  myPort.write(1);
28 }
```

The first thing you can do inside this method is to update the timestamp when the last data was read. This is to detect and recover from a connection timeout. Then read the bytes from the `frameBuffer` array which you can do with the help of the `readBytes()` method that returns the number of bytes read.


```
1 lastUpdate = millis();
2
3 // read the received bytes
4 myPort.readBytes(frameBuffer);
```

Then the frame buffer is translated into a ByteBuffer that allows for easy and safe access to the underlying bytes without having to worry about the array indices.

```
1 // Access raw bytes via ByteBuffer
2 ByteBuffer bb = ByteBuffer.wrap(frameBuffer);
```

Next we read the frame buffer and convert the bytes into pixel color values. The image gets constructed by sequentially filling the pixels array of the image. The conversion of the raw data is done with

`color()` and `Byte.toUnsignedInt()`.

```
1 int i = 0;
2
3 while (bb.hasRemaining())
4     // read 8-bit pixel
5     byte pixelValue = bb.get();
6
7     // set pixel color
8     myImage.pixels[i++] = color(pixelValue);
9 }
```

Once all the pixels have been updated, you need to tell the sketch to redraw the image. Additionally, you can send an acknowledgement back to the arduino sketch to ask it to send the

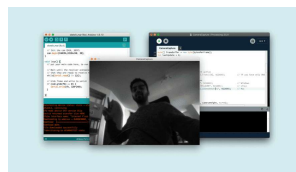
update the image with

`updatePixels()` and write `1` to the serial port for the acknowledgement.

```
1 myImage.updatePixels();
2
3 // Ensures that the new
4 shouldRedraw = true;
5
6 // Let the Arduino sketch
7 // and are ready for the
8 myPort.write(1);
```

5. Upload the Sketch

Select the right serial port on your IDE and upload the Arduino sketch to your Portenta H7. After a successful upload, run the `CameraViewer.pde` sketch in Processing. You should be able to see the rendered camera output on the Processing canvas.



Camera output on
Processing

Conclusion

In this tutorial you learnt how to capture the frames from your Portenta Vision Shield's Camera and to visualize the frames through Processing. This knowledge can be useful for you to build and

applications for both outdoor and indoor environments.

Complete Sketch

The `CaptureRawBytes.ino` Sketch.

```
1  #include "camera.h"
2
3  /*-----Uncomment the l
4
5  // #include "himax.h"
6  // HM01B0 himax;
7
8  #include "hm0360.h" //
9  HM0360 himax;
10
11 Camera cam(himax);
12 #define IMAGE_MODE CAM
13 FrameBuffer fb(320,240
14
15 unsigned long lastUpda
16
17 void setup() {
18     Serial.begin(921600)
19     //Init the cam QVGA,
20     cam.begin(CAMERA_R32
21 }
22
23 void loop() {
24     // put your main cod
25     if (!Serial) {
26         Serial.begin(92160
27         while (!Serial);
28     }
```

The `CameraViewer.pde` Sketch.

```
1  /*
2   This sketch reads a
3   from the Serial port
4   the window.
5   Use with the Example
6   This example code is
7  */
8
9  import processing.serial.*;
10 import java.nio.ByteBuffer;
11 import java.nio.ByteOrder;
12
13 Serial myPort;
14
15 // must match resolution
16 final int cameraWidth;
17 final int cameraHeight;
18 final int cameraBytesPerPixel;
19 final int cameraPixelCount;
20 final int bytesPerFrame;
21
22 PImage myImage;
23 byte[] frameBuffer = new byte[0];
24 int lastUpdate = 0;
25 boolean shouldRedraw = false;
26
27 void setup() {
28   size(640, 480);
29 }
```

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

